# ApproxLP: Approximate Credal Network Inference via Iterative Linearization

IBM Research

**Abstract**

ApproxLP is an approximate inference algorithm for credal networks that reformulates the marginal inference problem as a multilinear program and solves it via iterative linearization (coordinate descent). At each step, all local conditional distributions except one are fixed, reducing the optimization to selecting the best extreme point for the remaining variable. This produces *inner* bounds—the returned interval is guaranteed to be contained within the true probability interval. ApproxLP is dramatically faster than exact variable elimination while often finding the exact bounds on small to medium networks.

## 1 Preliminaries

### 1.1 Multilinear Programming Formulation

The marginal inference task in a credal network asks for:

$$\underline{P}(Z{=}z \mid \mathbf{Y}{=}\mathbf{y}) = \min_{P \in \operatorname{ext} K(\mathbf{X})} \frac{\sum_{\mathbf{x}'} \prod_i P(x_i \mid \pi_i)}{\sum_{z',\mathbf{x}'} \prod_i P(x_i \mid \pi_i)},$$

where the optimization is over all combinations of extreme points of the local credal sets. Using a tree decomposition and symbolic variable elimination (see `credal_ve.tex`), this can be reformulated as a multilinear program:

$$\begin{aligned} \text{minimize} \quad & f'_m(z), \\ \text{subject to} \quad & f'_m(Z) = t \cdot f_m(Z), \quad \sum_{z'} f'_m(z') = 1, \\ & f_i = \sum_{x_i} \prod \{f \in \text{bucket}_i\}, \quad i = 1, \ldots, m{-}1, \\ & P(X_i \mid \pi_i) \in K(X_i \mid \pi_i), \quad \forall X_i, \pi_i. \end{aligned} \tag{1}$$

The multilinearity arises because the bucket constraints involve products of the optimization variables $P(X_i \mid \pi_i)$ and the intermediate functions $f_i$.

### 1.2 The Linearization Idea

Solving the multilinear program (**??**) directly is expensive. The key insight of Antonucci et al. (2013, 2015) is to apply **coordinate descent**: fix all local distributions $P(X_j \mid \pi_j)$ for $j \neq i$ to specific values, then the program becomes **linear** in the remaining variable $P(X_i \mid \pi_i)$.

Since the optimal solution of a linear program over a polytope is attained at a vertex, and we already have the extreme points of each credal set $K(X_i \mid \pi_i)$ from LRS vertex enumeration, the linear optimization step reduces to:

*Try each extreme point of $K(X_i \mid \pi_i)$ and pick the one that optimizes the objective.*

No LP solver is needed—vertex enumeration suffices.

# 2 Algorithm

**Algorithm 1** ApproxLP: Coordinate Descent over Extreme Points

**Require:** Credal factors with extreme points, query $Z$, evidence $\mathbf{Y}{=}\mathbf{y}$, max iterations $T$, sense $\in \{\min, \max\}$

**Ensure:** Bound on $P(Z{=}z \mid \mathbf{Y}{=}\mathbf{y})$

▷ Initialization

**for** each variable $X_i$, each parent config $\pi$ **do**
  $P(X_i \mid \pi) \leftarrow \text{center}(\text{ext}\, K(X_i \mid \pi))$ ▷ Average of extreme points
**end for**
$p^* \leftarrow \text{VE}(\{P(X_i \mid \pi)\}, Z, \mathbf{y})$ ▷ Run standard VE with fixed distributions

▷ Coordinate Descent

**for** $t = 1, \ldots, T$ **do**
  *improved* $\leftarrow$ false
  **for** each variable $X_i$ **do**
    **for** each parent config $\pi$ of $X_i$ **do**
      $P_{\text{best}} \leftarrow P(X_i \mid \pi)$ ▷ Current incumbent
      **for** each vertex $v \in \text{ext}\, K(X_i \mid \pi)$ **do**
        $P(X_i \mid \pi) \leftarrow v$ ▷ Temporarily set to vertex
        $p_v \leftarrow \text{VE}(\{P(X_j \mid \pi_j)\}, Z, \mathbf{y})$ ▷ Evaluate objective
        **if** sense = min **and** $p_v(z) < p^*(z)$ **then**
          $p^* \leftarrow p_v, \quad P_{\text{best}} \leftarrow v, \quad$ *improved* $\leftarrow$ true
        **else if** sense = max **and** $p_v(z) > p^*(z)$ **then**
          $p^* \leftarrow p_v, \quad P_{\text{best}} \leftarrow v, \quad$ *improved* $\leftarrow$ true
        **end if**
      **end for**
      $P(X_i \mid \pi) \leftarrow P_{\text{best}}$
    **end for**
  **end for**
  **if** $\neg$ *improved* **then break** ▷ Converged
  **end if**
**end for**
**return** $p^*(z)$

## 2.1 Computing Both Bounds

Algorithm **??** is run twice:

- With sense = min to obtain $\underline{P}(Z{=}z \mid \mathbf{y})$.
- With sense = max to obtain $\overline{P}(Z{=}z \mid \mathbf{y})$.

Each run performs independent coordinate descent starting from the same initialization.

### 2.2 Standard VE with Fixed Distributions

The subroutine VE($\{P(X_i \mid \pi)\}, Z, \mathbf{y}$) runs classical variable elimination on the precise Bayesian network defined by the fixed distributions. This is a standard operation:

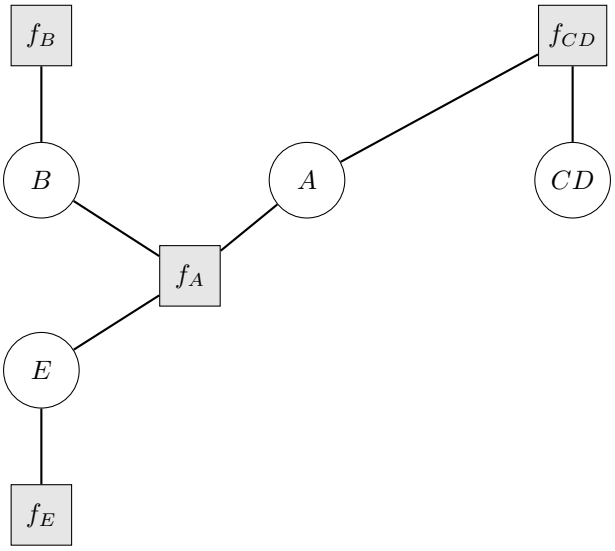
1. Build a factor for each variable: a numpy array over $\{X_i\} \cup \mathrm{Pa}(X_i)$ filled from $P(X_i \mid \pi)$.
2. Add indicator factors for evidence variables.
3. Eliminate all variables except $Z$ by combining and marginalizing.
4. Normalize and return $P(Z \mid \mathbf{y})$.

Since all distributions are fixed (no sets of functions), each VE run is very fast.

## 3 Detailed Example: Alarm Network

We trace ApproxLP on the alarm network (`alarm.lcn`) with variables $B$, $E$, $A$, $CD$ and the DAG $B \to A$, $E \to A$, $A \to CD$.

### 3.1 Factor Graph




### 3.2 Credal Factors (Extreme Points)

$f_B$**:** $K(B)$**.** $v_1 = [0.80, 0.20]$, $v_2 = [0.90, 0.10]$.

$f_E$**:** $K(E)$**.** $v_1 = [0.90, 0.10]$, $v_2 = [0.95, 0.05]$.

$f_A$**:** $K(A \mid B, E)$**.** 4 parent configs, 2 vertices each:

| Config | Vtx | $P(A{=}0)$ | $P(A{=}1)$ |
|---|---|---|---|
| $B{=}0, E{=}0$ | $v_1$ | 0.9556 | 0.0444 |
| $B{=}0, E{=}0$ | $v_2$ | 1.0000 | 0.0000 |
| $B{=}1, E{=}0$ | $v_1$ | 0.0000 | 1.0000 |
| $B{=}1, E{=}0$ | $v_2$ | 1.0000 | 0.0000 |
| $B{=}0, E{=}1$ | $v_1$ | 0.0000 | 1.0000 |
| $B{=}0, E{=}1$ | $v_2$ | 1.0000 | 0.0000 |
| $B{=}1, E{=}1$ | $v_1$ | 0.0000 | 1.0000 |
| $B{=}1, E{=}1$ | $v_2$ | 1.0000 | 0.0000 |

**$f_{CD}$: $K(CD \mid A)$.** $A{=}0$: 4 vertices; $A{=}1$: 11 vertices (see companion documents for full listing).

### 3.3 Query 1: $P(B{=}1)$, No Evidence

**Step 1: Initialization.** Each distribution is set to the center (average) of its credal set:

$$P(B) = \tfrac{1}{2}([0.80, 0.20] + [0.90, 0.10]) = [0.85,\ 0.15],$$
$$P(E) = \tfrac{1}{2}([0.90, 0.10] + [0.95, 0.05]) = [0.925,\ 0.075],$$
$$P(A \mid B{=}0, E{=}0) = \tfrac{1}{2}([0.9556, 0.0444] + [1.0, 0.0]) = [0.9778,\ 0.0222].$$

(Other parent configs for $A$ are similarly averaged.)

**Step 2: Initial evaluation.** Run VE with these fixed distributions:

$$P(B{=}1) = 0.1500 \quad \text{(the center value for } B\text{)}.$$

**Step 3: Coordinate descent (sense = min). Iteration 0:**
*Try variable $B$, config ():*

- Set $P(B) = v_1 = [0.80, 0.20]$, run VE: $P(B{=}1) = 0.2000$. Worse than $0.1500 \rightarrow$ reject.
- Set $P(B) = v_2 = [0.90, 0.10]$, run VE: $P(B{=}1) = 0.1000$. Better $\rightarrow$ **accept**. Update incumbent: $p^* = 0.1000$.

*Try variable $E$, config ():*

- $v_1 = [0.90, 0.10]$: VE gives $P(B{=}1) = 0.1000$. No improvement.
- $v_2 = [0.95, 0.05]$: VE gives $P(B{=}1) = 0.1000$. No improvement.

Since $B$ is a root node, $P(B)$ directly determines $P(B{=}1)$ regardless of other distributions. The algorithm correctly identifies $v_2 = [0.90, 0.10]$ as the minimizer in one iteration.

*Other variables ($A$, $CD$)* have no effect on $P(B{=}1)$ after $B$'s vertex is fixed.

**Iteration 1:** No improvement found $\rightarrow$ converged.

**Result (min):** $\underline{P}(B{=}1) = 0.1000$.

**Coordinate descent (sense = max).** Analogous: selecting $v_1 = [0.80, 0.20]$ gives $\overline{P}(B{=}1) = 0.2000$. Converges in 1 iteration.

**Final result:**

$$\boxed{P(B{=}1) \in [0.1000,\ 0.2000].}$$

### 3.4 Query 2: $P(A{=}1 \mid B{=}0, E{=}0)$

**Initialization.** With evidence $B{=}0, E{=}0$, the only relevant parent config for $A$ is $(B{=}0, E{=}0)$. The initial center distribution is:

$$P(A \mid B{=}0, E{=}0) = [0.9778,\ 0.0222].$$

**Initial evaluation.** VE with fixed distributions and evidence gives $P(A{=}1 \mid B{=}0, E{=}0) = 0.0222$.

**Coordinate descent (sense = min).** *Try A, config* $(B{=}0, E{=}0)$*:*

- $v_1 = [0.9556, 0.0444]$: VE gives $P(A{=}1) = 0.0444$. Worse $\rightarrow$ reject.
- $v_2 = [1.0000, 0.0000]$: VE gives $P(A{=}1) = 0.0000$. Better $\rightarrow$ **accept**.

Converged: $\underline{P}(A{=}1 \mid B{=}0, E{=}0) = 0.0000$.

**Coordinate descent (sense = max).** *Try A, config* $(B{=}0, E{=}0)$*:*

- $v_1 = [0.9556, 0.0444]$: $P(A{=}1) = 0.0444$. Better $\rightarrow$ **accept**.
- $v_2 = [1.0, 0.0]$: $P(A{=}1) = 0.0000$. Worse $\rightarrow$ reject.

Converged: $\overline{P}(A{=}1 \mid B{=}0, E{=}0) = 0.0444$.

**Final result:**

$$\boxed{P(A{=}1 \mid B{=}0, E{=}0) \in [0.0000,\ 0.0444].}$$

Both results match exact VE, confirming that ApproxLP finds the global optimum on this network.

## 4 Properties

**Inner bounds.** ApproxLP produces *inner* bounds: the returned interval $[\underline{P}, \overline{P}]$ is guaranteed to satisfy $\underline{P}_{\text{true}} \leq \underline{P}$ and $\overline{P} \leq \overline{P}_{\text{true}}$. This is because coordinate descent explores a subset of the feasible extreme point combinations.

**Monotonic convergence.** Each iteration either improves the objective or stops. The objective is bounded, so convergence is guaranteed in a finite number of steps.

**Local optima.** Coordinate descent may get stuck at a local optimum: the current combination of vertices may be locally optimal (no single vertex swap improves the objective) but globally suboptimal (a simultaneous swap of multiple vertices would improve it). In practice, this is rarely an issue for small to medium networks.

**Speed.** Each VE evaluation operates on a *precise* Bayesian network (single distributions, no sets of functions), which is orders of magnitude faster than credal VE. For the alarm network: ApproxLP takes 0.007 s vs. credal VE's 3.76 s—a 500× speedup.

## 5 Comparison with Other Methods

| Property | Exact VE | $\varepsilon$-VE | CTE | IBP | ApproxLP |
|---|---|---|---|---|---|
| Exact | Yes | No | Yes | No | No |
| Bound type | Tight | Outer | Tight | Outer | Inner |
| All marginals | No | No | Yes | No | No |
| Multi-valued | Yes | Yes | Yes | Yes | Yes |
| Speed (alarm) | 3.76 s | 0.009 s | 50 s | 0.01 s | 0.007 s |

ApproxLP and IBP produce complementary bounds: ApproxLP's inner bounds sit inside the true interval, while IBP's outer bounds contain it. Together they can bracket the true interval from both sides.

## 6 Complexity

Each coordinate descent iteration visits all variables, all parent configs, and all extreme points. For each vertex tried, a full VE is run on a precise BN.

- **Per VE evaluation**: $O(n \cdot k^{w^*})$ where $n$ is the number of variables, $k$ the max domain size, and $w^*$ the treewidth.
- **Per iteration**: $O\big(\sum_i \sum_\pi |\text{ext}\, K(X_i \mid \pi)|\big) \times O(n \cdot k^{w^*})$ VE evaluations.
- **Total**: $O(T \cdot V_{\text{total}} \cdot n \cdot k^{w^*})$ where $T$ is the number of iterations and $V_{\text{total}}$ is the total number of extreme points across all credal sets.

In practice, $T$ is typically 2–5 (fast convergence), making ApproxLP very efficient.

## 7 References


1. A. Antonucci, C.P. de Campos, D. Huber, M. Zaffalon. Approximating credal network inferences by linear programming. *ECSQARU*, 7958:13–25, 2013.
2. A. Antonucci, C.P. de Campos, D. Huber, M. Zaffalon. Approximate credal network updating by linear programming with applications to decision making. *IJAR*, 58:25–38, 2015.
3. C.P. de Campos, F.G. Cozman. Inference in credal networks using multilinear programming. *STAIRS*, 2004.
4. A.M. Lukatskii, D.V. Shapot. Problems in multilinear programming. *Comp. Math. Math. Phys.*, 41:638–648, 2000.
5. D.D. Mauá, F.G. Cozman. Thirty years of credal networks. *IJAR*, 126:133–157, 2020.